perplexity

Semester system architecture blueprint: OIDC IdP federation, custom domains, and integrated ed-tech app services in one spec

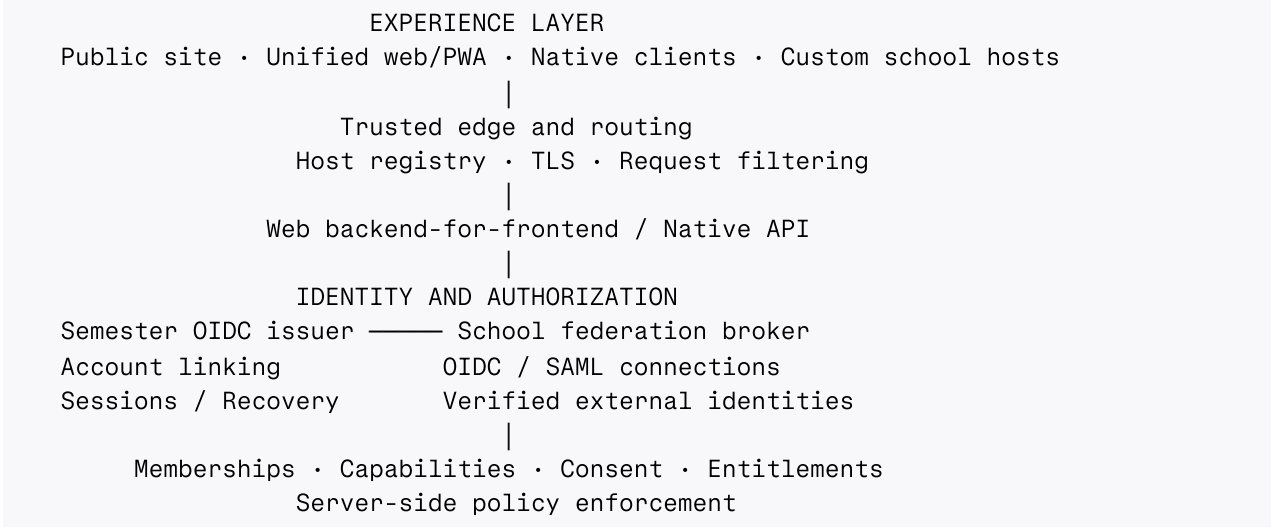
Computer
Semester product roadmap: phased milestones, security hardening, and app suite expansion across 4 quarters
Computer
Detail the OIDC federation and account mapping spec
Write out the Phase 1 through 3 rollout roadmap
Specify the custom domain routing and SSL automation
Sources
73

Semester should implement one architecture with three shared foundations: a canonical account and OIDC federation layer, verified custom-domain routing with automated TLS, and an integrated education app suite governed by the same authorization system.

The critical design principle is **separation**: one person can have one Semester Account, while each institution retains its own membership, permissions, records, and policy boundaries.

This specification expands the account-mapping and domain-automation details, then organizes phases 1–3 across four delivery quarters. It is a proposed implementation plan—not a claim of completed infrastructure. Quarter labels are relative to project kickoff; they are not promised calendar dates.

1. System architecture



```
                              |
                     EDUCATION SERVICES
Plan · Learning · Registration · Advising · Career · Campus
Mail · Calendar · Files · Community · Support · Administration
                              |
                     SHARED PLATFORM
Search · Education graph · Audit · Events · Notifications
          Policy-controlled AI · Integration reconciliation
                              |
                        DATA LAYER
Personal data scopes · Institution data scopes · Mail engine
          Private objects · Scoped indexes · Queues
```

Reconcile this with Semester's existing modular architecture. Do not create another account system, permissions engine, file service, or app shell. The supplied role-system specification explicitly requires shared canonical identity, membership, policy, and lifecycle systems.[1]

Proposed service boundaries

Boundary	Owns	Does not own
Identity	Authentication, external-identity mappings, sessions, OAuth grants	Official academic status
Membership/policy	Institutional relationships and resource authorization	User credentials
Domain control plane	Host ownership, mapping, certificate readiness, lifecycle	Membership authorization
Education modules	Domain workflows and authoritative-source handling	Independent login systems
Mail	Mailbox lifecycle, transport integration, mail access	Automatic authority over academic records
Shared platform	Audit, files, notifications, search, AI enforcement	Unrestricted access across tenants

Start with modules where possible. Extract services when isolation, operational requirements, or scaling evidence justify it.

2. OIDC federation and account-mapping specification

2.1 Identity authority

Before implementation, establish which existing system owns the canonical account identifier.

The architecture decision must specify:

- Current authentication authority.
- Immutable account identifier.
- Existing account/profile/membership relationships.
- Target OIDC implementation.
- External-identity mapping strategy.

- Migration, reconciliation, and rollback.
- How existing sessions and integrations are affected.

Use a maintained OIDC implementation. Do not write a new protocol engine or token cryptography merely to make the service “native.”

2.2 Identifier contract

```
account_id
  Immutable Semester identity.

external_identity
  Trusted connection + issuer + subject → account_id.

membership_id
  account_id + institution + lifecycle relationship.

username / email
  Mutable attributes; not identity join keys.
```

An OIDC provider’s `sub` identifies the authenticated subject within its issuer. Email is not a safe permanent account key; Google specifically documents that its email attribute can change while `sub` remains stable.[2]

For school federation, include the approved connection in the mapping namespace. This prevents ambiguous mappings when different institutional configurations use the same upstream identity infrastructure.

2.3 Proposed logical records

Reconcile these concepts with existing tables before creating anything.

Record	Essential fields
Account	`account_id`, lifecycle state, recovery configuration
Federation connection	`connection_id`, `tenant_id`, protocol, trusted issuer/entity ID, configuration version, approval state
External identity	`connection_id`, issuer, subject, `account_id`, linked/last-seen timestamps
Membership	`membership_id`, `account_id`, `tenant_id`, state, authoritative source, effective dates
Role assignment	Membership, capability/role reference, organizational scope, effective dates
Authentication transaction	Client, connection, redirect URI, state/nonce references, PKCE challenge, expiry
Session	Account, authentication method, assurance, expiry, revocation state
OAuth grant	Account, client, consented scopes, status
Security event	Actor, action, result, configuration version, correlation ID

Store secrets and private signing material in an appropriate secret/key-management system, not ordinary configuration rows.

2.4 First-party sign-in

```
Application
→ Semester authorization endpoint
→ authenticate user
→ issue one-time authorization code
→ code exchange with PKCE
→ validate token response
→ establish application session
→ resolve permitted workspaces
```

Requirements:

- Authorization Code flow with PKCE `S256`.
- Exact registered redirect URIs.
- Transaction-bound state and nonce handling.
- Signature, issuer, audience, expiry, and applicable nonce validation.
- Separate client registrations and token audiences.
- Refresh-token replay protections.
- No implicit or resource-owner-password flow.

RFC 9700 supplies the OAuth security baseline, including PKCE, redirect matching, refresh-token protections, and defenses against authorization-server mix-up.[3]

For browser clients, prefer a backend-for-frontend with server-held tokens and host-scoped, secure session cookies. Native clients use the system browser and secure device storage.

2.5 School federation

```
Select school
→ locate active approved connection
→ bind connection to authentication transaction
→ redirect to school IdP
→ validate callback against that connection
→ resolve external identity
→ resolve account
→ separately resolve school membership
→ establish appropriately assured session
```

Do not:

- Choose a trusted issuer based on an unvalidated callback parameter.
- Fetch arbitrary discovery URLs submitted by a user.
- Treat an email suffix as enrollment verification.

- Convert every upstream group into a powerful Semester role.
- Let personal authentication bypass a school's required assurance level.

For OIDC, configure trusted issuer and client relationships explicitly. For SAML, use maintained validation with signature, audience, destination, timing, and request-correlation checks.

2.6 Account-resolution decision table

Situation	Required behavior
External identity already mapped	Resolve its existing account
Unmapped identity; user authenticated to existing Semester Account	Offer explicit linking after proving both identities
Unmapped identity; no existing authenticated account	Create or provision an account only under approved policy
Same email appears on another account	Do not merge; use conflict-resolution workflow
Identity is mapped to a different account	Block silent reassignment; escalate verified recovery
Upstream issuer or subject changes	Run an approved identity-migration workflow
School membership ends	Remove that school's authority without deleting unrelated identity relationships

For initial rollout, avoid automatic account merges. If merges later become necessary, require proof of control, a data/ownership impact preview, approval, audit, and conflict handling.

2.7 Provisioning and authorization

Authentication, provisioning, and authorization are distinct:

- Authentication: who signed in.
- Provisioning: which account and institutional relationship exist.
- Authorization: what this person can do now.

School memberships can originate from an approved provisioning feed, invitation, or other institution-authorized workflow. Just-in-time creation should be limited to what the school explicitly permits.

Every consequential action evaluates current server-side state:

```
Account active?
Membership active?
Resource in permitted scope?
Capability granted?
Policy and consent satisfied?
Required assurance achieved?
Workflow and entitlement permit action?
```

Do not depend on stale role claims in long-lived tokens.

2.8 Offboarding and recovery

A school access termination must:

1. Disable the affected membership and grants.
2. Revoke relevant sessions or require reauthentication.
3. Invalidate authorization caches.
4. Remove search/file/AI access derived from that membership.
5. Preserve unrelated memberships and eligible personal data.
6. Apply institution-specific retention and export rules.

Semester's existing lifecycle specification already requires this scoped offboarding, credential revocation, and portability behavior.[1]

Recovery must not depend exclusively on the Semester mailbox being recovered.

3. Custom-domain routing specification

Let `D` represent Semester's verified primary domain.

Supported domain classes

Class	Example	Purpose
First-party service	`app.D`, `identity.D`, `mail.D`	Semester-owned services
Semester tenant subdomain	`school-slug.D`	Institution-branded entry point
Institution custom hostname	`learn.school.example`	Institution-controlled branding
Mail domain	Approved address domain	Email routing; separate activation workflow

Keep the OIDC issuer stable at `identity.D` initially. Institution branding does not require a separate issuer per school.

Domain registry

Proposed fields:

```
domain_binding_id
normalized_hostname
tenant_id or service_id
domain_class
verification_state
verification_challenge_reference
provider_binding_reference
certificate_state
routing_state
configuration_version
created_by / approved_by
```

```
verified_at / last_checked_at
suspended_at / removed_at
```

Enforce a globally unique active hostname binding. Normalize case and internationalized domain representations consistently, and reject malformed hosts, IP literals, unsupported wildcard patterns, and reserved service names.

Request routing

```
Incoming request
→ trusted edge terminates TLS
→ normalize and validate host
→ resolve active domain binding
→ assign server-side candidate tenant context
→ authenticate
→ verify membership and resource authorization
→ serve permitted workspace
```

Unknown, unverified, suspended, or removed hosts fail closed with a neutral error. Never fall back to a privileged "default institution."

Trust forwarded-host headers only from the known proxy chain.

Cross-domain login

A school custom host should not share Semester's account cookie directly.

Instead:

```
Custom host
→ registered OIDC login transaction
→ identity.D authenticates
→ exact approved callback on custom host
→ server-side code exchange
→ host-scoped application session
```

Register each callback through the verified-domain workflow. Do not use wildcard redirect URIs or arbitrary post-login destinations.

4. TLS certificate automation

"SSL automation" should mean automated modern TLS certificate issuance, renewal, monitoring, and recovery.

Recommended Vercel deployment option

If the verified Semester deployment uses Vercel, use its managed domain and certificate lifecycle rather than creating a second certificate automation system.

Vercel documents custom-domain provisioning, verification, and automatic certificate issuance. Its wildcard setup requires access to DNS challenges; current platform guidance describes nameservers or delegated certificate validation where supported. [4] [5]

This is a deployment option, not a claim that Semester's live project is configured that way.

Activation state machine

```
Requested
→ institution authority approved
→ ownership challenge issued
→ ownership verified
→ provider binding created
→ routing DNS verified
→ certificate pending
→ certificate issued
→ HTTPS and tenant-isolation probes pass
→ active
```

Semester should require an explicit ownership challenge for every institution-controlled custom hostname, even when the hosting provider does not demand one.

Show provider-generated DNS instructions. Do not hardcode a guessed CNAME or IP address.

Separate ownership proof from certificate validation

These checks serve different purposes:

- Semester ownership verification: establishes that the authorized institution controls the requested host.
- CA validation: allows the certificate authority to issue a certificate.
- Routing verification: proves requests reach the correct deployment.
- Authorization testing: proves the host cannot bypass tenant boundaries.

Passing one does not replace the others.

ACME standardizes automated certificate authorization and issuance; DNS-01 proves control through a designated TXT record. [6]

Wildcards

Use wildcard certificates only where operationally justified.

`*.D` covers first-level subdomains such as `school.D`, not `files.school.D`. Vercel explicitly documents that boundary and the need for DNS-challenge access.[7]

Do not require a university to move its entire DNS zone just to use one Semester hostname. Prefer an approved subdomain arrangement and supported validation delegation.

Renewal and monitoring

Monitor:

- Certificate expiry and renewal status.
- DNS drift and lost validation delegation.
- Provider verification state.
- HTTPS hostname and chain validation.
- Tenant mapping correctness.
- Removal or reassignment attempts.

Proposed operational targets, subject to validation:

- Renewal-failure escalation while substantial certificate validity remains.
- Daily active-domain health checks.
- Immediate incident escalation for a wrong-tenant mapping.
- A documented alternative first-party URL for custom-host outages.

These are design targets, not measured service commitments.

Removal and rollback

```
Suspend routing
→ revoke domain-specific login callbacks and sessions as appropriate
→ invalidate edge/application caches
→ remove provider binding
→ instruct owner to remove stale DNS
→ retain an audit tombstone
→ require fresh verification before reuse
```

Certificate issuance alone must never reactivate a removed domain.

5. Integrated app-suite expansion

All services use the same platform primitives, while retaining domain-specific authority.

Shared primitive	Connected services
Account and policy	Entire suite
Calendar	Courses, advising, campus, mail
Action Center	Assignments, applications, planning, support
Files and provenance	Study, portfolio, advising, attachments
Scoped search	Personal and institutional resources
Education graph	School/course/resource relationships
AI gateway	Permitted retrieval and bounded assistance
Audit and events	Controlled writes and downstream reconciliation

Semester Mail should use a qualified mail engine as the authoritative mailbox store, with JMAP-facing integration, private message/attachment storage, and scoped search. Semester stores provisioning, ownership, policy, and workflow references—not an unnecessary second copy of every mailbox. JMAP Core and Mail define synchronization state and mailbox/message/submission objects.[8] [9]

Mail-to-task or mail-to-calendar actions require source provenance and user approval. An email is not authority to change an official grade or enrollment record.

6. Four-quarter roadmap: phases 1–3

Security hardening continues through every quarter. A failed gate delays activation, not necessarily all unrelated development.

Quarter	Phase	Primary milestone	Activation boundary
Q1	Phase 1A	Identity reconciliation, domain foundations, policy hardening	Internal environments
Q2	Phase 1B	OIDC/federation pilot and verified custom-host pilot	Approved institution cohort
Q3	Phase 2	Mail pilot and integrated productivity workflows	Invitation-only users
Q4	Phase 3	Multi-school suite expansion and reviewed ecosystem access	Evidence-qualified staged release

Phase 1 — Identity, federation, and domain foundations

Q1: reconcile and harden

Accountable owner: platform/identity lead.

Deliver:

- Canonical identity decision and mapping inventory.
- Threat model and authorization matrix.
- Domain registry and ownership workflow.
- OIDC client registrations and secure session design.

- Recovery, linking, signing-key, and revocation runbooks.
- Existing RLS, definer, and AI-policy gap reconciliation.

Gate:

- No unexplained duplicate identities.
- Cross-tenant negative tests pass.
- Secrets and signing-key controls are verified.
- Existing risks have remediation or approved bounded exceptions.

The supplied architecture snapshot records unresolved security/tenancy issues, including a "NO-GO · in progress" phase. These are dated findings to reconcile—not a live status assertion. [10]

Q2: controlled federation and domain launch

Accountable owners: identity lead, institution IT owner, platform operations.

Deliver:

- One institution-approved federation connection.
- Safe account linking and membership provisioning.
- One custom-domain pilot with managed TLS.
- Scoped offboarding and recovery drills.
- Authentication and domain observability.

Gate:

- Wrong issuer/audience, replay, redirect abuse, and mapping collisions fail.
- Custom-host routing cannot cross tenants.
- Certificate issuance and failure recovery are tested.
- An institution signs off on configuration and lifecycle behavior.

Rollback: disable the affected federation/domain binding, revoke relevant sessions, and preserve a secure approved first-party access path.

Phase 2 — Mail and productivity integration

Q3: invitation-only mail and connected workflows

Accountable owner: messaging lead; supporting owners: security, privacy, support, and operations.

Deliver:

- Mail-engine selection and provider-exit contract.
- Idempotent mailbox provisioning and reconciliation.

- JMAP synchronization, drafts, submission, and search.
- SPF/DKIM/DMARC and delivery monitoring.
- Export, restore, retention, abuse, and appeals.
- User-approved mail-to-task/calendar integration.

Gate:

- Unauthorized mailbox/blob access and send-as attempts fail.
- Retries do not duplicate mailbox provisioning.
- Stale JMAP states trigger safe resynchronization.
- Restore and export/import drills pass.
- Support and abuse coverage are staffed.
- Personal mail is not exposed through school-admin privileges.

Rollback: stop new provisioning or outbound submission selectively; preserve existing messages, safe inbound delivery, and export access.

Phase 3 — Multi-school suite and ecosystem expansion

Q4: expand proven workflows

Accountable owners: product/platform leads and institutional domain owners.

Deliver:

- Personal versus school workspace switching.
- Scoped cross-school planning and search.
- Shared Calendar, Files, Action Center, and career evidence.
- Graduation and institution-offboarding continuity.
- Additional approved custom domains.
- A small reviewed partner OIDC pilot.
- Native-client/offline lifecycle tests.

Gate:

- One school cannot inspect another’s records or private workspace.
- Expired permissions cannot be bypassed offline.
- Partner grants are limited, consented, revocable, and auditable.
- Core journeys meet accessibility and support requirements.
- Migration and rollback evidence exists for every newly enabled domain.

Rollback: disable individual apps, partner grants, or domain bindings independently without deleting accounts or trapping user data.

7. Release evidence and ownership

Maintain one readiness record per capability:

```
Capability
Accountable owner
Canonical implementation
Dependencies and blockers
Designed / implemented / tested / deployed
Institution-authorized / production-enabled
Test and operational evidence
Activation approval
Rollback and portability procedures
Last verification date
```

The first execution artifact should be the canonical identity-and-domain reconciliation register: existing account identifiers, trusted connections, domain ownership, current deployment configuration, authorization gaps, and migration decisions. That register determines how to implement this specification without duplicating Semester's existing systems or treating proposed architecture as completed work.

⁂

1. The Main Semester design system (2) copy/handoff/specifications/role-system-execution-prompt.pdf
2. https://developers.google.com/identity/openid-connect/openid-connect
3. https://www.rfc-editor.org/info/rfc9700/
4. https://vercel.com/docs/platforms/multi-tenant-platforms/configuring-domains
5. https://vercel.com/docs/platforms/multi-tenant-platforms/limits
6. https://www.rfc-editor.org/info/rfc8555/
7. https://vercel.com/docs/platforms/multi-tenant-platforms/concepts
8. https://www.rfc-editor.org/info/rfc8620/
9. https://www.rfc-editor.org/info/rfc8621/
10. The Main Semester design system (2) copy/uploads/Semester build Architecture.pdf
11. https://www.rfc-editor.org/subjects/acme/
12. https://www.rfc-editor.org/info/rfc9799/
13. https://www.rfc-editor.org/info/rfc9447/
14. https://vercel.com/docs/platforms/multi-tenant-platforms/quickstart
15. https://vercel.com/docs/domains/working-with-domains
16. https://vercel.com/solutions/multi-tenant-saas
17. https://github.com/rcbj/oauth2-oidc-debugger/issues/253
18. https://github.com/vercel/vercel/discussions/7739
19. https://www.rfc-editor.org/subjects/oauth/

20. https://oauth.net/2/oauth-best-practice/